

AI-BASED COLLEGE ACADEMIC ASSISTANT

An Agentic AI Academic Support System using RAG, LangChain and LangGraph

PROJECT REPORT

TEAM MEMBERS

Sanmathi R Shetty

Sannidhya BS

Shaarwari Balegadde

Sharadhi H K

Sathwik S L

GitHub Repository

https://github.com/Sharadhihk/Agentic_AI_College_Academic_Assistant

Table of Contents

1. Abstract
2. 1. Introduction
3. 2. Problem Statement
4. 3. Objectives
5. 4. Scope
6. 5. Technology Stack
7. 6. System Architecture
8. 7. Document Processing and RAG
9. 8. LangChain Integration
10. 9. LangGraph Workflow
11. 10. Personalized Study Planner
12. 11. Conversational Context
13. 12. Tool Integration
14. 13. Streamlit Interface
15. 14. Testing and Validation
16. 15. Representative Test Cases
17. 16. Results
18. 17. Limitations
19. 18. Future Enhancements
20. 19. Conclusion
21. 20. Team
22. 21. GitHub Repository
23. References

Abstract

The AI-Based College Academic Assistant is a student-focused academic support system that brings college information, document-based question answering and personalized study planning into one place. The project combines a Large Language Model (LLM), Retrieval-Augmented Generation (RAG), LangChain and LangGraph. College documents are converted into searchable representations so that students can ask questions in natural language instead of manually searching through several files.

The assistant is designed to do more than answer questions. It can retrieve information from academic documents, summarize content, understand conversational follow-up questions, create personalized study plans, modify existing plans when a student's schedule changes, and perform calculation and date-related operations. LangGraph coordinates these capabilities through an intent-analysis and routing workflow, while a review step checks whether generated answers are supported by retrieved context. A Streamlit interface provides the user-facing layer. The implementation is modular so that the RAG pipeline, agentic workflow, planner, tools and interface can be developed and improved independently.

1. Introduction

Students frequently need information about attendance, examinations, syllabus requirements, internships, projects and academic regulations. Although this information is usually available, it may be spread across different documents and difficult to locate quickly. A normal chatbot can provide conversational answers, but a general model may not know institution-specific rules and can produce unsupported information.

This project addresses that gap by combining retrieval with an LLM. Instead of asking the model to answer only from its general knowledge, the system first retrieves relevant content from college documents and supplies that content to the model. The assistant is therefore designed to answer from the available academic sources and to explicitly say when the requested information cannot be found.

The project also treats academic assistance as a workflow. Different requests need different actions: a document question needs retrieval, a calculation needs a deterministic tool, and a study-plan request needs scheduling logic. LangGraph is used to coordinate these paths while preserving conversational state.

2. Problem Statement

Students often have to search through multiple academic documents to find small but important pieces of information. Syllabus documents, regulations, examination instructions, internship guidelines and project instructions may be stored separately. Generic AI assistants also do not automatically know the rules of a particular college and may respond with information that is not supported by official documents.

The goal of this project is to develop a single academic assistant that can retrieve college-specific information, answer questions conversationally, summarize documents, create personalized study plans and update those plans when student requirements change. The system should also avoid confidently answering questions for which relevant college information is unavailable.

3. Objectives

The project aims to:

- Build a college-document question-answering system using RAG.
- Process documents into searchable chunks and embeddings.
- Store embeddings in a vector database and retrieve relevant content.
- Integrate a Groq-based LLM using LangChain.
- Use LangGraph to coordinate analysis, retrieval, generation, review, planning and tools.
- Support conversational follow-up questions.
- Provide document summarization.
- Generate personalized study plans using subjects, exam dates, difficulty and available study time.
- Allow students to modify and recalculate their plans.

- Provide calculator and date-related tools.
- Handle unknown questions without inventing college-specific information.
- Provide a simple Streamlit user interface.

4. Scope

The current prototype focuses on academic assistance using college-provided documents such as syllabi, regulations, examination guidelines, internship information, project instructions and FAQs. The assistant is intended to support students rather than replace official college communication.

The study-planning component accepts subjects, examination dates, subject difficulty and daily study hours. The planner distributes time in 0.5-hour blocks, gives greater weight to difficult subjects and gives additional weight to the day before an examination as a revision period. Students can change examination dates, available hours, individual-day hours, subjects and difficulty, after which the schedule is rebuilt.

5. Technology Stack

The project is implemented primarily in Python. LangChain is used for LLM, prompt and tool integration. LangGraph provides workflow orchestration and state management. Groq provides the LLM interface. The RAG pipeline uses PDF loading, text splitting, HuggingFace embeddings and Chroma as the vector store. Streamlit is used for the user interface.

The architecture is deliberately modular. The RAG component exposes a retriever, the LangGraph component consumes the LLM and retriever, the planner contains deterministic scheduling logic, and the Streamlit layer handles presentation and user interaction.

6. System Architecture

The system follows a layered workflow. A student submits a request through the Streamlit interface. The request enters LangGraph, where the analysis node identifies the user's intent and rewrites follow-up questions into standalone questions when required.

Academic questions and summaries are routed to the retrieval branch. The retriever searches the Chroma vector database and returns relevant college-document chunks. These chunks are passed to the Groq LLM through a prompt that instructs it to answer only from the supplied context. The generated response then passes through a review node. If the reviewer identifies a grounding problem and a retry is available, the graph returns to generation with reviewer feedback.

Study-plan requests use the planner branch. Calculation and date requests use deterministic tools. Casual conversation is handled by a separate branch. MemorySaver provides thread-based state so conversation history and the study profile can persist across turns.

7. Document Processing and RAG

The knowledge base is created from college documents. PDF content is extracted and divided into smaller chunks. Each chunk is converted into an embedding and stored in Chroma. When a student asks a question, the retriever searches these embeddings and returns the most relevant chunks.

The LangGraph retrieval node converts retrieved documents into an internal representation containing the text and source name. Source names are retained so that final responses can identify the documents used.

RAG is important because it gives the LLM current, institution-specific context rather than relying only on information learned during model training. The generation prompt explicitly tells the assistant to use only the supplied context and to state that the information could not be found when the context does not contain the answer.

8. LangChain Integration

LangChain connects the LLM, prompts, retriever and deterministic tools. Separate prompt templates are used for academic question answering and document summarization. The existing RAG retriever is passed into the LangGraph application instead of creating a second retrieval system.

The project also uses LangChain tools for calculations and date operations. The calculator uses an AST-based evaluator rather than Python `eval()`, so arbitrary Python code is not executed. The date tools provide operations such as finding the number of days until a date and adding days to a starting date.

9. LangGraph Workflow

LangGraph is the main orchestration layer. The assistant state stores messages, the original question, the standalone question, intent, retrieved documents, answer, review status, retry count, study profile, study plan and structured UI inputs.

The implemented nodes are `analyze`, `retrieve`, `no_answer`, `generate`, `review`, `tool_node`, `plan_create`, `plan_modify`, `plan_show` and `chitchat`. The `analyze` node supports intents including question answering, summarization, plan creation, plan modification, plan display, calculation, date calculation and casual conversation.

The graph uses conditional routing. Retrieval is followed by generation when documents are available and by the no-answer branch when retrieval returns nothing. Generated answers are sent to review. A failed review can loop back to generation, creating a simple verification cycle. The graph can also be compiled with `MemorySaver` so that a thread ID maintains conversation state between turns.

10. Personalized Study Planner

The study planner is deterministic Python logic rather than an LLM-generated schedule. A profile contains subjects, exam dates, difficulty, daily study hours and optional per-day overrides.

Study time is allocated according to difficulty and the number of days remaining. The day immediately before an exam receives increased weight and is marked as revision. Allocations use 0.5-hour blocks. This makes the schedule predictable and easier to recalculate.

The planner supports setting exam dates, changing daily hours, changing a particular day's hours, adding or removing subjects and changing difficulty. After a change, the profile is updated and the complete schedule is rebuilt. This is safer than manually modifying individual schedule entries.

11. Conversational Context

The assistant supports follow-up questions by maintaining recent conversation history. During analysis, the LLM is asked to rewrite the latest message as a standalone question using the previous conversation. This allows questions such as "What if I fall short of it?" to be interpreted using the earlier topic.

The same state mechanism is used for study planning. Once a student has created a plan, later requests can refer to that existing plan without repeating all subject and exam information. The thread ID is used to keep each conversation separate.

12. Tool Integration

Three deterministic tools are included: `calculator`, `days_until` and `add_days`. The calculator supports common arithmetic operations and protects against unsupported expressions, excessively large exponentiation and invalid operations. The date tools accept dates in YYYY-MM-DD format and return readable results.

The tool node uses the LLM to understand what information is required and then calls the appropriate deterministic function. This keeps arithmetic and date calculations reliable instead of relying on the model to perform them internally.

13. Streamlit Interface

A Streamlit interface has been prepared as the user-facing component. It is intended to provide a simple conversational experience while also supporting study-plan interaction.

The LangGraph implementation exposes an `ask(app, text, thread_id=...)` interface and supports structured study-plan inputs and plan changes. This allows UI forms and buttons to send structured information directly to the planner rather than requiring the LLM to parse every field. The final application connects the existing UI to the compiled graph using the same LLM and retriever.

14. Testing and Validation

The LangGraph and planner implementation was tested using a stub LLM and stub retriever so that the workflow could be validated without depending on a live API. The tests cover graph structure, the RAG path, follow-up rewriting, unknown-question handling, review retry, calculator operations, date operations, study-plan creation, plan modification, structured UI edits and safety checks.

The recorded test run completed 47 out of 47 checks successfully. The tests verified that the expected graph nodes existed, the Mermaid graph could be generated, the RAG and follow-up paths worked with the stub retriever, the unknown-question branch behaved correctly, and the planner handled daily hours, subject difficulty, revision days and modifications.

The sample calculator test produced 8.2727. Safety tests also checked invalid or unsafe expressions. These results validate the workflow and deterministic logic; they do not replace testing with the actual Groq model and the real college retriever.

15. Representative Test Cases

The intended application behaviour can be illustrated through the following cases:

1. Direct academic question: retrieve relevant college content and generate a grounded answer with sources.
2. Follow-up question: use conversation history to resolve references such as “it” or “that”.
3. Unknown question: return a clear not-found response instead of inventing college-specific information.
4. Calculation: route an arithmetic request to the calculator.
5. Date calculation: route a date query to the date tool.
6. Study-plan creation: accept subjects, exam dates, difficulty and daily hours.
7. Study-plan modification: change an examination date or available time and rebuild the schedule.
8. Plan display: retrieve the student's current plan from persisted state.

16. Results

The project demonstrates a complete agentic workflow in which retrieval, language generation, deterministic tools and personalized planning are coordinated through a single graph. The separation of responsibilities is one of the main strengths of the design: the vector store handles retrieval, the LLM handles natural-language understanding and generation, LangGraph handles routing and state, and the planner and tools handle deterministic operations.

The available stub-based validation completed 47/47 checks successfully. The study planner correctly handled the tested daily-hour constraints, gave harder subjects greater weight, marked the day before an examination as revision and rebuilt the schedule after changes. The RAG workflow also supported direct questions, follow-ups and an explicit unknown-information branch in the test environment.

17. Limitations

The prototype has several limitations. Unknown-question handling depends on the retriever returning no documents for irrelevant questions. A similarity retriever may still return weakly related content, so its score threshold should be tuned using the real college documents.

The stub tests use controlled model outputs. A real LLM may format intent or date information differently, although the graph contains keyword and regular-expression fallbacks for common cases. The planner also does not currently

model every real-life constraint such as class schedules, travel, sleep, holidays or total workload capacity.

The assistant should not be treated as the final authority for important academic decisions. Students should verify critical rules with official college sources or offices when necessary.

18. Future Enhancements

Future versions can improve retrieval through tuned thresholds, hybrid search, reranking and page-level citations. The study planner can be extended with class timetables, holidays, preferred study periods, completed topics and progress tracking.

Additional tools could include calendar integration, assignment deadline tracking, timetable lookup and campus-service information. The interface could also include a visual study calendar, progress dashboard and reminders. Stronger evaluation of retrieval relevance, answer grounding and real-model behaviour would further improve reliability.

19. Conclusion

The AI-Based College Academic Assistant demonstrates how an LLM can be turned into a practical academic support system rather than a simple chatbot. By combining RAG, LangChain and LangGraph, the project connects college documents, conversational interaction, deterministic tools and personalized study planning.

The system is designed around a simple idea: students should be able to ask questions naturally while the assistant uses the right source or operation behind the scenes. A student can ask about college information, continue with a follow-up, request a summary, perform a calculation, create a study plan and change that plan as circumstances change.

The modular implementation provides a strong foundation for future development. With further testing using the actual deployed UI and retriever, improved retrieval thresholds and richer student preferences, the system can be extended into a more complete academic support platform.

20. Team

The project was developed by five team members: Sanmathi R Shetty, Sannidhya BS, Shaarwari Balegadde, Sharadhi H K and Sathwik S L. The project is organized into modular components so that document retrieval, agentic workflow, study planning, tools, testing and the user interface can be developed collaboratively.

Specific contribution percentages or role assignments should only be added if they have been formally agreed upon by the team.

21. GitHub Repository

The complete source code and project files are maintained in the following repository:

https://github.com/Sharadhihk/Agentic_AI_College_Academic_Assistant

The repository serves as the central location for the application code, supporting files and project documentation.

References

1. LangChain documentation and project resources.
2. LangGraph documentation and project resources.
3. Chroma documentation and project resources.
4. HuggingFace documentation for embedding models.
5. Groq API documentation.
6. Streamlit documentation.
7. Project source repository: https://github.com/Sharadhihk/Agentic_AI_College_Academic_Assistant
8. Project implementation files: graph.py, planner.py, study_tools.py and the associated test report.